Universidade Estadual de Campinas

MC656 — Engenharia de Software

Ágora

Atividade 4 — Projeto e Arquitetura de Software

Equipe Ágora

Lucas Mantovani Boaro	RA 281303
Gabriel Saia Schade	RA 281202
Bruno Amaral Salles de Moraes	RA 281129
Fernando de Almeida Maciel Penido	RA 281180
Mateus Pachela Garcia	RA 259554

Campinas, outubro de 2026

Sumário

1 Arquitetura do sistema **2**
1.1 Escopo e estado da implementação 2
1.2 Diagrama C4 de componentes 2
1.3 Componentes, responsabilidades e relações 2
1.3.1 Componentes 2
1.3.2 Relações representadas pelas setas 3
1.4 Estilos arquiteturais 4
1.5 Decisões arquiteturais 5
1.5.1 Decisão 1: concentrar os módulos em uma aplicação Express 5
1.5.2 Decisão 2: usar armazenamento local e em memória nesta versão 5
1.5.3 Decisão 3: concentrar integrações externas em componentes específicos . . 5

2 Padrões de projeto **6**
2.1 Strategy: interpretação da proposta orçamentária por exercício 6
2.2 Adapter: provedor de busca de informações em rede 6

3 Code smells e refatoração **7**
3.1 Ocorrência 1: Duplicate Code na validação do cadastro (refatorada) 7
3.2 Ocorrência 2: Data Clumps nos tipos de usuário 7

1 Arquitetura do sistema

1.1 Escopo e estado da implementação

O recorte deste documento é o backend Node.js, Express e TypeScript da ramificação `main` no commit `7fbb42d` (tag `a2-v1.0.0`). A interface React atual consome apenas a geração de pautas. As outras funcionalidades existem como rotas HTTP; propostas e votos ficam em memória, enquanto usuários e resultados Gorilla ficam em arquivos JSON locais. A extração de orçamento contempla a proposta de 2020 do Instituto de Computação.

1.2 Diagrama C4 de componentes

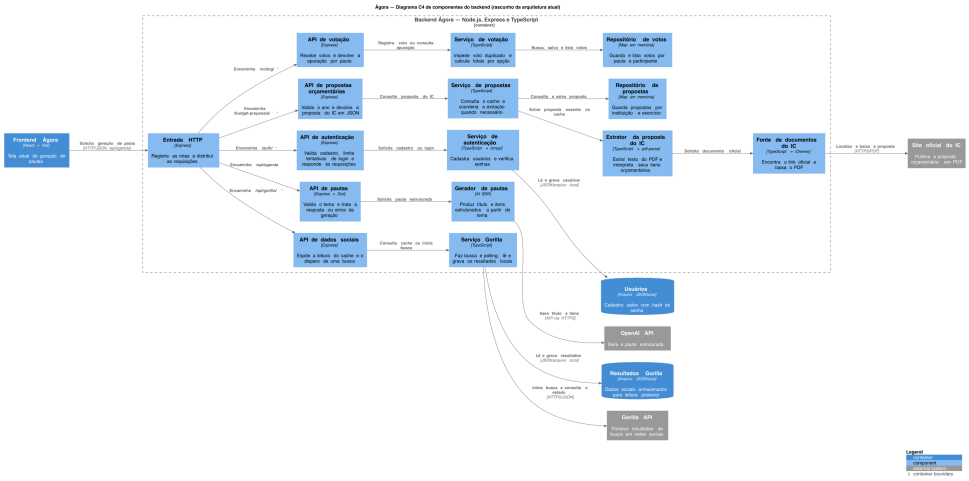


Figura 1: Diagrama C4 de componentes do backend.

1.3 Componentes, responsabilidades e relações

O diagrama C4 de nível 3 detalha os componentes do backend Ágora. Cada seta aponta do componente que solicita uma operação para aquele que a executa ou para o dado acessado. As respostas retornam pelo caminho inverso.

1.3.1 Componentes

- **Entrada HTTP:** configura a aplicação Express e encaminha as requisições para as APIs de autenticação, propostas orçamentárias, votação, pautas e dados sociais.
- **API de autenticação:** recebe as requisições de cadastro e login, valida os dados de entrada e chama o Serviço de autenticação.
- **Serviço de autenticação:** cadastra usuários e verifica senhas com `bcrypt`. Lê e grava os registros em um arquivo JSON local.
- **API de propostas orçamentárias:** recebe `GET /budget-proposals/ic/:year`, valida o ano solicitado e chama o Serviço de propostas.
- **Serviço de propostas:** consulta uma proposta já extraída no Repositório de propostas. Caso não a encontre, chama o Extrator da proposta do IC e guarda o resultado obtido.
- **Repositório de propostas:** mantém as propostas por instituição e exercício em um `Map` em memória. Seus dados se perdem quando o servidor reinicia.

- **Extrator da proposta do IC:** recebe o PDF, extrai seu texto com `pdf-parse` e interpreta os itens orçamentários. A implementação analisada contempla a proposta de 2020.
- **Fonte de documentos do IC:** localiza o PDF na página oficial do Instituto de Computação com auxílio de `Cheerio` e o baixa por HTTPS para o Extrator.
- **API de votação:** recebe votos e consultas de apuração por pauta, valida a entrada e chama o Serviço de votação.
- **Serviço de votação:** impede votos duplicados do mesmo `voterId` na mesma pauta, registra votos válidos e calcula os totais por opção.
- **Repositório de votos:** guarda e lista votos por pauta e participante em um `Map` em memória. Os registros não sobrevivem ao reinício do servidor.
- **API de pautas:** recebe `POST /api/agenda`, valida o contexto com `Zod` e solicita a geração de uma pauta. É a API utilizada pela interface React nesta versão.
- **Gerador de pautas:** usa o AI SDK e a API da OpenAI para produzir um título e itens de pauta estruturados a partir do tema informado.
- **API de dados sociais:** expõe `GET /api/gorilla` para ler resultados salvos e `POST /api/gorilla/fetch` para iniciar uma busca. Encaminha ambas as operações ao Serviço Gorilla.
- **Serviço Gorilla:** inicia buscas na API Gorilla, acompanha sua conclusão e grava os resultados em um arquivo JSON local. Também lê esse arquivo em consultas posteriores.

1.3.2 Relações representadas pelas setas

As cinco setas que saem da **Entrada HTTP** representam o encaminhamento das rotas `/auth`, `/budget-proposals`, `/voting`, `/api/agenda` e `/api/gorilla`. Todas pertencem à mesma aplicação Express; as setas não indicam servidores separados.

Cadastro e login.

1. Um cliente chama uma rota `/auth/*`. A Entrada HTTP encaminha a requisição à API de autenticação.
2. A API valida a entrada e solicita cadastro ou verificação de credenciais ao Serviço de autenticação.
3. O serviço lê ou grava o arquivo local de usuários. No cadastro, armazena o hash da senha; no login, compara a senha informada com o hash armazenado.

Consulta da proposta orçamentária do IC.

1. Um cliente chama `GET /budget-proposals/ic/:year`. A API de propostas valida o ano e chama o Serviço de propostas.
2. O serviço consulta o Repositório de propostas. Se encontrar a proposta, devolve os dados já armazenados.
3. Caso contrário, chama o Extrator da proposta do IC. O Extrator pede o PDF à Fonte de documentos do IC, que o localiza e baixa do site oficial.
4. O Extrator interpreta o PDF; o Serviço de propostas guarda o resultado no repositório e a API o devolve em JSON. A extração implementada atualmente é a da proposta de 2020.

Registro e apuração de votos.

1. Um cliente chama uma rota `/voting/*`. A Entrada HTTP encaminha a requisição à API de votação.
2. Para registrar o voto, a API valida `voterId` e a opção escolhida. O Serviço de votação consulta o Repositório de votos para verificar duplicidade e, quando permitido, grava o voto.
3. Para apurar, o serviço lista os votos da pauta e calcula os totais por opção. O `voterId` vem da requisição; não há seta para o Serviço de autenticação porque essa integração não existe na implementação analisada.

Geração de pautas.

1. A interface React envia `POST /api/agenda`. A Entrada HTTP encaminha a requisição à API de pautas.
2. A API valida o tema e solicita uma resposta estruturada ao Gerador de pautas.
3. O Gerador consulta a API da OpenAI e devolve título e itens. A API envia o resultado em JSON para a interface.

Busca e leitura de dados sociais.

1. Um cliente chama `POST /api/gorilla/fetch`. A API de dados sociais solicita uma nova coleta ao Serviço Gorilla.
2. O serviço inicia a busca na API Gorilla, consulta seu estado até a conclusão e grava os resultados em um arquivo JSON local.
3. Em `GET /api/gorilla`, a API solicita os resultados já salvos. O serviço lê o arquivo e responde sem iniciar outra busca externa.

Os arquivos de usuários e de resultados Gorilla aparecem fora da borda do backend para explicitar as dependências de armazenamento local; eles não são serviços independentes. O site oficial do IC, a API da OpenAI e a API Gorilla são sistemas externos. Os repositórios em `Map` permanecem dentro da borda porque fazem parte do processo Express.

1.4 Estilos arquiteturais

Cliente–servidor. O sistema separa a interface React do backend Express. O cliente apresenta a tela de geração de pautas e envia uma requisição HTTP com JSON para `POST /api/agenda`; o backend valida a entrada, executa a operação e devolve uma resposta JSON. As demais rotas também podem ser chamadas por clientes HTTP, embora ainda não estejam integradas à interface atual. No diagrama, essa relação aparece na seta que liga o frontend à Entrada HTTP. O site do IC, a API da OpenAI e a API Gorilla são dependências externas acessadas pelo servidor, não diretamente pela interface.

Monólito modular. O backend é uma única aplicação Node.js/Express. O arquivo de composição da aplicação registra os grupos de rotas de autenticação, propostas orçamentárias, votação, pautas e dados sociais no mesmo processo. Cada domínio tem componentes com responsabilidades próprias, mas eles não são serviços independentes: suas chamadas internas ocorrem dentro da aplicação. A borda do backend no diagrama C4 representa esse único contêiner; os componentes desenhados dentro dela detalham sua organização interna. Essa estrutura permite desenvolver os recursos por módulo sem introduzir comunicação de rede entre eles.

Separação em camadas de responsabilidade. Nos módulos de propostas, votação e dados sociais, as rotas e os controladores tratam a interface HTTP, os serviços coordenam as operações e as regras, e os repositórios ou fontes tratam o acesso aos dados. Por exemplo, a API de propostas chama o Serviço de propostas, que consulta o Repositório de propostas e, quando necessário, chama o Extrator; o Extrator usa a Fonte de documentos para obter o PDF. As setas do diagrama refletem essas dependências. Essa separação não é uniforme em todo o código: o Serviço de autenticação acessa diretamente o arquivo JSON, e a rota de pautas e seu gerador estão no mesmo módulo. Portanto, o projeto usa uma organização em camadas de responsabilidade, sem implementar uma arquitetura em camadas rígida em todos os recursos.

1.5 Decisões arquiteturais

1.5.1 Decisão 1: concentrar os módulos em uma aplicação Express

Justificativa. A equipe implementou autenticação, propostas, votação, pautas e dados sociais como grupos de rotas de um único backend. Para o estágio atual do projeto, essa composição permite compartilhar a entrada HTTP e integrar novos módulos sem configurar serviços ou comunicação de rede adicionais.

Consequências e trade-offs. A estrutura reduz a complexidade de execução e facilita chamadas internas entre componentes. Em contrapartida, os módulos compartilham o mesmo processo e ciclo de implantação: uma falha ou uma operação muito custosa pode afetar outras rotas, e não é possível escalar cada módulo de forma independente sem alterar a arquitetura. A separação por pastas e responsabilidades ajuda a manter o código organizado, mas, por si só, não oferece o isolamento de serviços separados.

1.5.2 Decisão 2: usar armazenamento local e em memória nesta versão

Justificativa. O Repositório de propostas e o Repositório de votos usam `Map` em memória; usuários e resultados Gorilla são gravados em arquivos JSON locais. Esses mecanismos permitem disponibilizar os fluxos atuais sem configurar um banco de dados e, nos módulos de propostas e votação, mantêm o acesso aos dados atrás de repositórios usados pelos serviços.

Consequências e trade-offs. As propostas extraídas e os votos desaparecem quando o processo reinicia. Os arquivos JSON preservam dados entre reinícios na mesma instalação, mas dependem do sistema de arquivos local e não constituem um armazenamento compartilhado entre várias instâncias. Assim, a solução simplifica a implementação inicial, porém limita durabilidade, concorrência e escalabilidade. Além disso, o `voterId` é recebido diretamente na requisição de votação: o módulo ainda não consulta a autenticação para vincular o voto a um usuário cadastrado.

1.5.3 Decisão 3: concentrar integrações externas em componentes específicos

Justificativa. A Fonte de documentos do IC localiza e baixa o PDF oficial; o Extrator interpreta seu conteúdo; o Gerador de pautas faz a chamada à OpenAI; e o Serviço Gorilla consulta a API externa e salva seus resultados. Localizar essas chamadas nos respectivos componentes mantém as rotas voltadas ao tratamento das requisições e torna explícitas as dependências externas mostradas no diagrama.

Consequências e trade-offs. O backend passa a depender da disponibilidade, das credenciais e dos formatos fornecidos por terceiros. A consulta da proposta depende da página e do PDF do IC, cuja estrutura pode mudar; a geração de pautas depende da chave e da resposta da OpenAI; a coleta Gorilla depende de sua API e pode exigir espera pelo término da busca. Em troca, o restante de cada fluxo trabalha com dados já obtidos ou estruturados por esses componentes, o que delimita onde alterações nas integrações precisam ser tratadas.

2 Padrões de projeto

Os dois padrões a seguir respondem a limitações observadas no código e ligadas a pressupostos já definidos para o produto na Atividade 1. Cada aplicação tem issue e Pull Request próprios, identificados com o label `Atividade4`.

2.1 Strategy: interpretação da proposta orçamentária por exercício

Problema identificado. O Extrator da proposta do IC (`IcBudgetProposalExtractor`) tinha o exercício fixo no código: rejeitava qualquer ano diferente de 2020 com `if (year !== 2020)` e chamava diretamente a função `parseIcBudgetProposal2020`. O pressuposto P4 da Atividade 1 estabelece que os documentos de origem não são padronizados entre exercícios e que a extração precisa tolerar variações de estrutura. Assim, cada nova proposta, com seu próprio layout de PDF, exigiria alterar o extrator e acumular nele a lógica de vários formatos.

Padrão escolhido e justificativa. O *Strategy* separa um algoritmo que varia (a interpretação do texto de cada exercício) do componente que o utiliza (o fluxo de extração, que é o mesmo para todos os anos: obter o PDF, extrair o texto, interpretar e montar a proposta). Foi criada a interface `BudgetProposalParser`, com o exercício atendido (`year`) e a operação `parse(text)`. O parser existente passou a ser a estratégia `icBudgetProposal2020Parser`, sem alteração em sua lógica. O extrator recebe a lista de estratégias disponíveis e, em tempo de execução, escolhe a que corresponde ao ano pedido; quando não há estratégia, responde ao ano como não suportado antes de acessar o site do IC. O padrão é adequado porque a variação já é conhecida e recorrente (um formato por exercício) e o fluxo ao redor dela é estável.

Consequências. Suportar a proposta de outro exercício passa a exigir apenas a escrita e o registro de uma nova estratégia, sem modificar o extrator (princípio aberto/fechado). Cada parser pode ser testado isoladamente com o texto do seu PDF, e o comportamento de `GET /budget-proposals/ic/2020` permaneceu o mesmo. Como contrapartida, o módulo ganha uma interface e um registro de estratégias a manter, e a escolha continua baseada apenas no ano: novos contextos institucionais da Atividade 1 exigirão estender o critério de seleção.

Rastreabilidade. Issue #36; Pull Request #39.

2.2 Adapter: provedor de busca de informações em rede

Problema identificado. O Serviço Gorilla (`GorillaService`) concentrava, em um único componente, o protocolo da API Gorilla (URL, cabeçalho `x-api-key`, requisição inicial e consulta periódica do estado até `completed`) e a persistência dos resultados em JSON. O pressuposto P5 da Atividade 1 define a busca em rede como dependente de um serviço externo, "que tratamos como fornecedor substituível e não como parte do núcleo". No código, entretanto, trocar o fornecedor exigiria reescrever o serviço. Além disso, o método `fetchAndSave` recebia cinco parâmetros apenas para viabilizar testes, e a consulta de estado era um laço `while (true)` sem limite: se a busca externa não terminasse, a requisição ficaria presa indefinidamente.

Padrão escolhido e justificativa. O *Adapter* converte a interface de um componente externo na interface esperada pelo sistema. Foi definida a interface `SocialSearchProvider`, com a operação `search(query)`, que devolve os resultados no formato do Ágora (`SocialSearchResults`, o mesmo já exposto em `GET /api/gorilla`). A classe `GorillaSearchAdapter` implementa essa interface e encapsula todo o protocolo da API Gorilla, incluindo um número máximo de consultas de estado. O `GorillaService` passou a depender apenas da interface: solicita a busca ao provedor

e persiste o resultado. O padrão é adequado porque o problema é exatamente a incompatibilidade entre o protocolo de um fornecedor específico e a operação de que o sistema precisa.

Consequências. Um novo fornecedor de busca em rede passa a exigir apenas uma nova implementação de `SocialSearchProvider`, sem alterações no serviço, no controlador ou nas rotas. O serviço pode ser testado com um provedor falso, sem rede nem chave de API, e `fetchAndSave` passou de cinco para dois parâmetros. O limite de consultas também elimina a espera indefinida. Em contrapartida, há uma camada adicional de tipos e de mapeamento, e os formatos da API Gorilla e do Ágora precisam ser mantidos separados à medida que a integração evoluir. As rotas e respostas HTTP não mudaram.

Rastreabilidade. Issue #37; Pull Request #40.

3 Code smells e refatoração

A análise usou o catálogo de *code smells* do Refactoring.Guru. Foram identificadas duas ocorrências de tipos distintos, ambas no módulo de autenticação; a primeira foi selecionada para refatoração.

3.1 Ocorrência 1: Duplicate Code na validação do cadastro (refatorada)

Local e smell. `backend/src/modules/auth/routes.ts`, middleware `validateRegistrationData`, e `backend/src/modules/auth/service.ts`, função `registerUser`. Smell: *Duplicate Code*.

Justificativa. As mesmas regras de cadastro (campos obrigatórios, idade não negativa e tamanho mínimo da senha) estavam escritas duas vezes. As cópias já haviam divergido: o middleware não validava o formato do e-mail nem o gênero, que só eram verificados no serviço, e o serviço gerava o *hash* `bcrypt` da senha antes de validar os dados, executando uma operação custosa para entradas que seriam rejeitadas.

Impacto esperado. Com as regras duplicadas, qualquer nova exigência de cadastro precisa ser lembrada em dois arquivos, e uma atualização feita em apenas um deles produz respostas diferentes conforme o ponto de entrada. Centralizar as regras reduz esse risco e torna a validação testável isoladamente.

Refatoração aplicada. Foi aplicada a refatoração *Extract Function*: as regras passaram para `validateRegistration`, em `backend/src/modules/auth/validation.ts`, usada pelo middleware e pelo serviço. O serviço agora valida antes de ler o arquivo de usuários e de gerar o *hash*. Para garantir a preservação do comportamento, o primeiro commit do Pull Request adicionou testes de caracterização da rota de cadastro, executados com sucesso sobre o código anterior e mantidos sem alteração após a refatoração: respostas 400 com as mensagens `All fields are required`, `Age must be a positive number` e `Password must be at least 6 characters long`, e 400 para gênero e e-mail inválidos. A única diferença observável é que, para gênero e e-mail inválidos, a mensagem genérica `Failed to register user` foi substituída pelo motivo da recusa; o código de status continua 400, e o login não foi alterado.

Rastreabilidade. Issue #38; Pull Request #41.

3.2 Ocorrência 2: Data Clumps nos tipos de usuário

Local e smell. `backend/src/modules/auth/types.ts`, interfaces `UserRegistration` e `UserSaved`, e os usos desses campos em `backend/src/modules/auth/service.ts`. Smell: *Data Clumps*.

Justificativa. O mesmo grupo de cinco campos (`name`, `email`, `institution`, `age` e `gender`) aparece repetido nas duas interfaces, que diferem apenas na senha (`passwordPlain` ou `passwordHash`). O grupo reaparece campo a campo na verificação de tipo `isUserSavedArray` e na montagem do objeto salvo em `registerUser`. Esses dados sempre andam juntos e representam um mesmo conceito, o perfil do usuário, que não tem nome próprio no código.

Impacto esperado. Acrescentar ou alterar um atributo do perfil, como um identificador institucional para restringir a participação às consultas, exige editar esses trechos de forma coordenada, e esquecer um deles gera inconsistência silenciosa entre o que é recebido e o que é gravado. A refatoração indicada é extrair um tipo `UserProfile` e definir os dois tipos atuais a partir dele, com a respectiva informação de senha. Esta ocorrência não foi refatorada nesta atividade.